

Final Project

This document is the report for the final project of the course CI/CI DevOps - COMP.SE.140.

Implemented optional features

- Static analysis included in the pipeline: `eslint` and `pylint` for Python and JS code analysis. The pipeline will fail if there are any errors reported by either of these linter.
- Deployment to external server: the code is deployed to an CSC VM instance, accessible through a public IP: `http://86.50.169.142`. The nginx HTML interface is available at `http://86.50.169.142:8198/` and the API Gateway is available at `http://86.50.169.142:8197/`

Instructions for examiner to test the system.

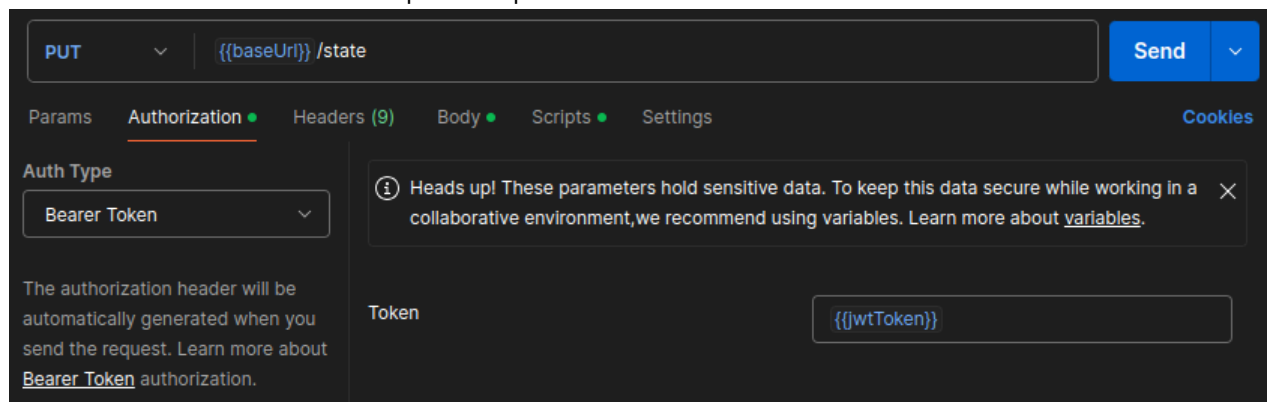
For this implementation, **the examiner should be able to test the system using the public IP without needing to clone the repository**. Here's the detailed instructions on that matter.

- The examiner can test the nginx interface at `http://86.50.169.142:8198/`. Since this is a public IP, it should be accessible from anywhere in the world (which is a bad practice, but it works here). The login credentials is `username` for the username and `password` for the password (same as the nginx exercise). This should work similarly to the previous exercise, since I did not include logging and monitoring to the system.
- The new REST API system can be tested with `curl` or Postman using the public IP `http://86.50.169.142:8197/`. Again, since it is a public IP, anyone should be able to send requests to it, and the system should work.
- Session management and login is done using JWT tokens. For that reason, it is highly recommended to test the REST API with Postman, since it is easier to use the post-request script system to extract the JWT token from the response for login. You can extract the token like this:

```
const token = pm.response.headers.get("Authorization")?.replace("Bearer ", "");

if (token) {
  pm.environment.set("jwtToken", token);
}
```

And the token can be used in sub-sequence request as an environment variable in the Authorization tab of Postman:



This is the preferred method for testing, since it is easier to setup and use.

- Alternatively, you can use `curl` to test the API, but it would be a little bit trickier since we need to provide the login credentials in the first request, and then extract the token for latter requests. Here is how it can be done:

```
TOKEN=$(curl -X PUT "http://86.50.169.142:8197/state" \
-H "Content-Type: text/plain" \
-u "username:password" \
-d "INIT" \
-i | grep -i "Authorization:" | sed 's/^Authorization: Bearer //')
```

This sends a POST request to set the system state as INIT, then extract the token. We can test if we grabbed the token by running `echo $TOKEN`. Sub-sequence requests can be sent without login credentials:

```
curl -X PUT "http://86.50.169.142:8197/state" \
-H "Content-Type: text/plain" \
-H "Authorization: Bearer $TOKEN" \
-d "RUNNING"
```

Other requests can be sent similarly without needing to re-login, using the same JWT token.

Data about the platform used in development

Hardware:

- Processor: Intel Core i5 - 10210U CPU
- CPU Architecture: x86_64
- Memory: 16GB DDR4
- Storage: 512GB SSD

Operating System:

- Name: Linux Mint 21.3 Cinnamon
- Linux Kernel Version: e.g., 5.15.0-130-generic

Development Tools:

- Docker Version: 27.3.1, build ce12230
- Docker Compose Version: 2.29.7
- Git Version: 2.34.1

Description of the CI/CD pipeline

- Since I'm working on the project alone, I did not use Git's branching in development much. All the work was done within the `project` branch of both the GitHub and GitLab repositories.
- The pipeline runs with GitLab runner, which is hosted in a CSC VM instance so it's up 24/7. The instance has a public floating IP address which I also allow connection via ports `:8197` and `:8198`, so that when the pipeline is finished, the project is hosted there and accessible through the internet.
- The pipeline builds the project with `docker`, using `node`, `python`, `nginx`, and `redis` images.
- The pipeline also has a static analysis step with `pylint` and `eslint`. If the analyzers find any errors in the code, the pipeline fails and those errors have to be fixed. This ensures code quality is always good.
- Testing was done using Postman's `newman` package, running via `node` runtime. I built the test using Postman's GUI, and the test suite was exported as `json` files and can be run in the pipeline.
- The test cases mimic a full run of the new REST API interface. From starting a new session with `PUT /state INIT`, to changing the state to `RUNNING`, and test the behavior of the system while it is paused and not. All the new endpoints of the API Gateway are tested with `newman` unit test.
- The pipeline then "deploy" the system to the VM instance with `docker compose up` so that it is up and running, accessible 24/7 through the public floating IP and the provided ports `:8197` and `:8198`. The system runs in Docker on the VM instance, and can be brought down manually by SSH into the VM and shutdown the containers with `docker compose down`.
- I did not implement monitoring to the system, except for the mandatory `/run-log` endpoint in the new APIs. It is also possible to see the system's statistics through the nginx interface, or through the `/request` endpoint of the API Gateway.
- With the way the GitLab runners are setup, new pipeline runs will be automatically triggered whenever there are changes pushed to the repository. The system will be built, lint, tested, and deployed to the VM instance. This process can be monitored through the GitLab's pipeline web GUI.

Example runs of the pipeline

Here are the logs for the linting and testing phases of the pipeline. First, the failed cases:

- Failing tests

```
- PUT /state - INIT (no JWT parsing)
  PUT http://localhost:8197/state [errored]
  read ECONNRESET
  27. Response is OK
  28. Response set state to INIT

- PUT /state - RUNNING (old token)
  PUT http://localhost:8197/state [errored]
  socket hang up
  30. Response is forbidden
  31. Response require login again

- PUT /state - SHUTDOWN
  PUT http://localhost:8197/state [errored]
  socket hang up
  33. Response is OK
  34. Response SHUTDOWN services
```

	executed	failed
iterations	1	0
requests	10	10
test-scripts	20	0
prerequest-scripts	12	0
assertions	24	24
total run duration: 619ms		
total data received: 0B (approx)		

```
[131m # [39m[131m failure [39m[131m detail [39m
[99m [39m[99m [39m[99m [39m[99m [39m
01. Error
    read ECONNRESET
    at request
    inside **
[99m [39m[99m [39m[99m [39m[99m [39m
02. AssertionError
    Response is OK
    expected PostmanResponse{...(5) } to have property 'code'
    at assertion:0 in test-script
    inside "PUT /state - INIT"
    [39m[99m
[99m [39m[99m [39m[99m [39m[99m [39m
03. AssertionError
    Response set state to INIT
    the given combination of arguments (undefined and string) is invalid for this assertion. You can use an array, a map, an object, a set, a string, or a weakset instead of a string
    at assertion:1 in test-script
    inside "PUT /state - INIT"
    [39m[99m
[99m [39m[99m [39m[99m [39m[99m [39m
04. Error
    socket hang up
    at request
    inside **
[99m [39m[99m [39m[99m [39m[99m [39m
05. AssertionError
    Response is OK
    expected PostmanResponse{...(5) } to have property 'code'
    at assertion:0 in test-script
    inside "PUT /state - PAUSED"
    [39m[99m
[99m [39m[99m [39m[99m [39m[99m [39m
06. AssertionError
    Response set state to PAUSED
```

- Failing lint

- `eslint` failed

```
1 Running with gitlab-runner 17.7.0 (3153ccc6)
2 on csc-ubuntu-vm t3_5Qq-MF, system ID: s_21ce97b7c4c7
3 Preparing the "shell" executor
4 Using Shell (bash) executor...
5 Preparing environment
6 Running on comp-se-140-vm...
7 Getting source from Git repository
8 Fetching changes with git depth set to 20...
9 Reinitialized existing Git repository in /home/ubuntu/builds/t3_5Qq-MF/0/vgkhng/comp-se-140/.git/
10 Checking out 8c3d2ec7 as detached HEAD (ref is project)...
11 Skipping Git submodules setup
12 Executing "step_script" stage of the job script
13 $ cd service1
14 $ npm install
15 added 216 packages, and audited 217 packages in 21s
16 41 packages are looking for funding
17 run 'npm fund' for details
18 2 moderate severity vulnerabilities
19 To address all issues, run:
20 npm audit fix
21 Run 'npm audit' for details.
22 $ npx eslint -c eslint.config.mjs *.js
23 /home/ubuntu/builds/t3_5Qq-MF/0/vgkhng/comp-se-140/service1/app.js
24 103:12 error 'error' is defined but never used no-unused-vars
25 119:12 error 'error' is defined but never used no-unused-vars
26 /home/ubuntu/builds/t3_5Qq-MF/0/vgkhng/comp-se-140/service1/auth.js
27 2:7 error 'Redis' is assigned a value but never used no-unused-vars
28 40:32 error 'Buffer' is not defined no-undef
29 ✖ 4 problems (4 errors, 0 warnings)
30 Cleaning up project directory and file based variables
31 ERROR: Job failed: exit status 1
```

- `pylint` failed

```
1 Running with gitlab-runner 17.7.0 (3153ccc6)
2 on csc-ubuntu-vm t3_5Qq-MF, system ID: s_21ce97b7c4c7
3 Preparing the "shell" executor
4 Using Shell (bash) executor...
5 Preparing environment
6 Running on comp-se-140-vm...
7 Getting source from Git repository
8 Fetching changes with git depth set to 20...
9 Reinitialized existing Git repository in /home/ubuntu/builds/t3_5Qq-MF/0/vgkhng/comp-se-140/.git/
10 Checking out b751f026 as detached HEAD (ref is project)...
11 Removing service1/node_modules/
12 Skipping Git submodules setup
13 Executing "step_script" stage of the job script
14 $ cd service2
15 $ which pylint
16 /home/ubuntu/.local/bin/pylint
17 $ pylint **/*.py
18 ***** Module app
19 app.py:7:0: E0401: Unable to import 'flask' (import-error)
20 -----
21 Your code has been rated at 6.43/10 (previous run: 6.43/10, +0.00)
22 Cleaning up project directory and file based variables
23 ERROR: Job failed: exit status 1
```

And the passing cases:

- Passing tests

```

→ PUT /state - RUNNING
PUT http://localhost:8197/state [200 OK, 254B, 8ms]
✓ Response is OK
✓ Response set state to RUNNING

→ GET /state when RUNNING
GET http://localhost:8197/state [200 OK, 233B, 6ms]
✓ Response is OK
✓ Get the current state: RUNNING

→ GET /request
GET http://localhost:8197/request [200 OK, 1.17kB, 502ms]
✓ Response is OK
✓ Content-Type is text/plain
✓ Response body has Service 1
✓ Response body has Service 2

→ GET /run-log
GET http://localhost:8197/run-log [200 OK, 314B, 7ms]
✓ Response is ok
✓ Content-Type is text/plain
✓ Response body is not empty
✓ Each line matches the expected log format

→ PUT /state - INIT (no JWT parsing)
PUT http://localhost:8197/state [200 OK, 432B, 18ms]
✓ Response is OK
✓ Response set state to INIT

→ PUT /state - RUNNING (old token)
PUT http://localhost:8197/state [403 Forbidden, 287B, 8ms]
✓ Response is forbidden
✓ Response require login again

→ PUT /state - SHUTDOWN
PUT http://localhost:8197/state [200 OK, 434B, 8ms]
✓ Response is OK
✓ Response SHUTDOWN services

```

	executed	failed
iterations	1	0
requests	10	0
test-scripts	20	0
prerequisite-scripts	12	0
assertions	24	0
total run duration: 1056ms		
total data received: 1.24kB (approx)		
average response time: 65ms [min: 6ms, max: 502ms, s.d.: 146ms]		

```

[]32;1m$ echo "Compose down after running tests..."[]0;m
Compose down after running tests...
[]32;1m$ docker compose down[]0;m
Container comp-se-140-nginx-1 Stopping
Container comp-se-140-redis-1 Stopping

```

- Passing lint

- `eslint` passed

```

1 Running with gitlab-runner 17.7.0 (3153ccc6)
2 on comp-se-140-vm t3_AwHyZ1, system ID: s_b1e5195034b2
3 Preparing the "shell" executor
4 Using Shell (bash) executor...
5 Preparing environment
6 Running on comp-se-140-vm...
7 Getting source from Git repository
8 Fetching changes with git depth set to 20...
9 Reinitialized existing Git repository in /home/ubuntu/builds/t3_AwHyZ1/0/vgkhng/comp-se-140/.git/
10 Checking out 723b656c as detached HEAD (ref is project)...
11 Skipping Git submodules setup
12 Executing "step_script" stage of the job script
13 $ cd service1
14 $ npm install
15 added 216 packages, and audited 217 packages in 22s
16 41 packages are looking for funding
17   run `npm fund` for details
18 2 moderate severity vulnerabilities
19 To address all issues, run:
20   npm audit fix
21 Run `npm audit` for details.
22 $ npx eslint -c eslint.config.mjs *.js
23 Cleaning up project directory and file based variables
24 Job succeeded

```

- `pylint` passed

```

10 Checking out 723b656c as detached HEAD (ref is project)...
11 Removing service1/node_modules/
12 Skipping Git submodules setup
13 Executing "step_script" stage of the job script
14 $ cd service2
15 $ pip install -r requirements.txt
16 Defaulting to user installation because normal site-packages is not writeable
17 Requirement already satisfied: flask in /home/ubuntu/.local/lib/python3.10/site-packages (from -r requirements.txt (line 1)) (3.1.0)
18 Requirement already satisfied: pylint in /home/ubuntu/.local/lib/python3.10/site-packages (from -r requirements.txt (line 2)) (3.3.3)
19 Requirement already satisfied: click>=8.1.3 in /home/ubuntu/.local/lib/python3.10/site-packages (from flask->-r requirements.txt (line 1)) (8.1.8)
20 Requirement already satisfied: itsdangerous>=2.2 in /home/ubuntu/.local/lib/python3.10/site-packages (from flask->-r requirements.txt (line 1)) (2.2.0)
21 Requirement already satisfied: blinker>=1.9 in /home/ubuntu/.local/lib/python3.10/site-packages (from flask->-r requirements.txt (line 1)) (1.9.0)
22 Requirement already satisfied: Werkzeug>=3.1 in /home/ubuntu/.local/lib/python3.10/site-packages (from flask->-r requirements.txt (line 1)) (3.1.3)
23 Requirement already satisfied: Jinja2>=3.1.2 in /home/ubuntu/.local/lib/python3.10/site-packages (from flask->-r requirements.txt (line 1)) (3.1.5)
24 Requirement already satisfied: astroid<=3.4.0-dev0,>=3.3.8 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (3.3.8)
25 Requirement already satisfied: mccabe<0.8,>=0.6 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (0.7.0)
26 Requirement already satisfied: dill>=0.2 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (0.3.9)
27 Requirement already satisfied: tomkit>=0.10.1 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (0.13.2)
28 Requirement already satisfied: platformdirs>=2.2.0 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (4.3.6)
29 Requirement already satisfied: isort!=5.13.0,<6,>=4.2.5 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (5.13.2)
30 Requirement already satisfied: toml>=1.1.0 in /home/ubuntu/.local/lib/python3.10/site-packages (from pylint->-r requirements.txt (line 2)) (2.2.1)
31 Requirement already satisfied: typing-extensions>=4.0.0 in /home/ubuntu/.local/lib/python3.10/site-packages (from astroid<=3.4.0-dev0,>=3.3.8->pylint->-r requirements.txt (line 2)) (4.12.2)
32 Requirement already satisfied: MarkupSafe>=2.0 in /home/ubuntu/.local/lib/python3.10/site-packages (from Jinja2>=3.1.2->flask->-r requirements.txt (line 1)) (3.0.2)
33 $ which pylint
34 /home/ubuntu/.local/bin/pylint
35 $ pylint **/*.py
36 -----
37 Your code has been rated at 10.00/10
38 Cleaning up project directory and file based variables
39 Job succeeded

```

Full runs and jobs can be found in the GitLab repository's pipelines section: <https://compse140.devops-gitlab.rd.tuni.fi/vgkhng/comp-se-140/-/pipelines>

Reflections

Main learning

- How Docker is used in real-world application and deployment. So far I used Docker mostly to setup a development environment for working within a team. Setting up and using Docker in production and deployment was a new experience to me.
- VM usage in CI/CD practices. So far I mostly used cloud providers' pipeline runners (GitHub action, BitBucket pipeline) that require minimal setup. But by setting up a VM instance an running GitLab runner, I learned how these runners work in the background to complete jobs. I also learned some networking knowledge by deploying to the VM instance, too.
- Building, debugging, and testing a CI/CD pipeline. There are many things that could, and will, go wrong in setting up a CI/CD pipeline, and it is crucial to know how to debug it and get it working.
- What nginx is, and how it is used as a web server, load-balancer, and reverse proxy for the system services.
- Learning some new technologies for software development, such as Redis, implementing JWT token, and revising my skills in Linux commands.

Worst difficulties

1. One thing caused me the most trouble was that I did not use `docker compose build --no-cache` initially. Because of that, the containers did not use the latest version of the code for the build, causing bugs that I was not able to solve initially. After some searching, I was able to use `docker exec -it <container-id> /usr/bin/bash` to access the bash shell of the containers to check for the code versions, thus solving my problem.
2. The Gitlab runner couldn't access the Docker daemon in the VM's docker user group -> Docker did not work correctly in the pipeline initially. To solve that problem, I followed this StackOverflow thread: <https://stackoverflow.com/questions/68081163/preparation-failed-got-permission-denied-while-trying-to-connect-to-the-docker>
3. Setting up the deployment to the CSC VM instance. The documentation of the CSC Pouta was hard to navigate, and there were no examples of how to do things, e.g., setting up public ports for the floating IP group.
4. When the GitLab server failed without me knowing about it (the man-in-the-middle warning), it took a lot of time for me to try and figure out what went wrong. I even tried to re-create the VM instance from scratch to see if any of my

configurations were wrong but that did not work either. I gave up in the end, and the issue resolved itself the following day.

Things that should have been done differently

- I am not sure about my implementation of session management with JWT. The project requirements did not describe much about login and authorization, so I went with this stateless approach that seemed the most sensible to me at time. However, I am curious to see if there are any simpler, easier to implement approach to this problem.
- I did not have much experience working with state management and Redis, did not know of something better and would love to learn about them more. I think my solution was a little bare-bone, and implementing something more atomic, with timestamp checking and monitoring for logs would be much better.
- I am concern about the scalability of the system. Right now, only service 1 has a load-balancer between 3 instances, but everything else only have 1 instance. Since I do not know about the read and write capability of Redis, I fear that if there are too many requests to the system, Redis might not be able to keep up. Implementing multiple cache that manage the states and session of the services might be a good idea.

Amount of hours used

Roughly 60 to 70 hours in total.